

## LECTURE 25

# Virtual Memory and Demand Paging

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

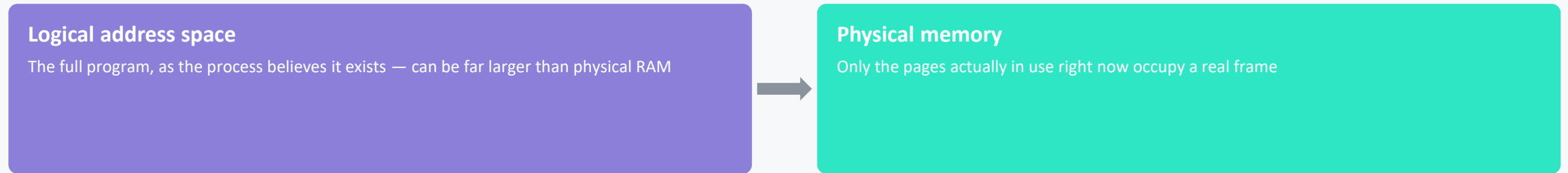
# In This Lecture

- What virtual memory is, and the three concrete benefits it buys an operating system
- **Demand paging**: loading a page only when it is first referenced, never speculatively
- **Page-fault handling**: the full step-by-step sequence, from CPU reference to restarted instruction
- **Effective Access Time (EAT)** with page faults, and why even a tiny fault rate can devastate performance
- **Copy-on-write (COW)**: how `fork()` avoids copying an address space it may never need

# Virtual Memory: Background and Benefits

**Virtual memory** separates a process's logical address space from the physical memory frames that back it. A page table entry can now be marked **invalid** — this page exists logically, but has no frame assigned to it right now.

A large logical address space, a smaller physical reality



**Virtual memory is an illusion maintained cooperatively.** The process never knows which pages are resident — hardware and OS conspire to keep the right frame behind every valid reference.

# Three Benefits of Separating Logical From Physical Memory

- **A program can be larger than physical memory.** Only the portion actively being used needs a frame, so logical size is no longer bounded by installed RAM.
- **More programs can run simultaneously.** Each process occupies frames only for its active portion, so far more processes fit in the same physical memory at once.
- **Less I/O is needed to load or swap a program.** A process doesn't have to be loaded or swapped in full, so disk I/O spent moving programs in and out of memory drops sharply.

# Demand Paging

**Demand paging** loads a page only when it is first referenced, never in advance, never speculatively.

- **Pure demand paging** starts a process with zero pages in memory.
- The very first instruction fetch is, by construction, a reference to a page that isn't resident — it immediately triggers a page fault, pulls that one page in, and lets execution continue.
- Every other page the process touches is loaded the same way, one fault at a time, purely in response to actual need.
- Pages the process never touches during its entire run are never loaded at all — not a wasted byte of I/O or memory.

**Why “pure” demand paging starts with nothing resident.** Real systems sometimes load a few pages eagerly (e.g. the entry point). “Pure” demand paging is the strict version used to reason about the mechanism cleanly: nothing loads until a reference proves it's needed.

# Page-Fault Handling: Trap and Decision

## 1. CPU references a page

Logical address generated by the running instruction

## 2. Page table marks it invalid

No frame currently backs this page — hardware traps to the OS

## 3. OS checks: valid reference, or illegal?

Was this address ever a legitimate part of the process's address space?

### 3a. Illegal access → abort the process

The reference was never valid — e.g. a stray pointer outside the address space

### 3b. Valid, just not yet loaded → continue

The page genuinely belongs to the process; it's just never been brought in

# Page-Fault Handling: Resolution

## 4. Find a free frame

From the free-frame list, or by running a page-replacement algorithm if none are free

## 5. Schedule a disk read

Read the page's contents in from backing store into the chosen frame

## 6. Update the page table

Mark the entry valid, and record which frame now holds it

## 7. Restart the faulting instruction

Re-execute from the very instruction that faulted — this time, the reference succeeds

**Step 7 restarts the instruction, not just the memory reference.** The entire faulting instruction — including any earlier operand fetches that already succeeded — is re-executed from the start. Safe because nothing has a side effect until the instruction completes.

# Effective Access Time (EAT) With Page Faults

$$\text{EAT} = (1 - p) \times \text{memory access time} + p \times \text{page fault service time}$$

where  $p$  is the page fault rate — the fraction of memory references that fault — and page fault service time is dominated by disk I/O (seek, rotational latency, transfer time; Lecture 28).

Worked example — memory access time 200 ns, page-fault service time 8 ms (8,000,000 ns)

| Quantity                | Value                         |
|-------------------------|-------------------------------|
| Memory access time      | 200 ns                        |
| Page fault service time | 8 ms = 8,000,000 ns           |
| Page fault rate $p$     | 0.001 (1 in 1,000 references) |

**EAT =  $0.999 \times 200 + 0.001 \times 8,000,000 = 199.8 + 8,000 = 8,199.8$  ns.** Roughly 8.2 microseconds — about 41× slower than the fault-free 200 ns baseline, even though 999 of every 1,000 references never fault at all.

# EAT: Sensitivity to the Fault Rate

At  $p = 0.0001$  (one in ten thousand):  $EAT = 0.9999 \times 200 + 0.0001 \times 8,000,000 = 199.98 + 800 = 999.98$  ns — still roughly 5× slower than 200 ns, from a fault rate ten times lower than the example above.

EAT at  $m_a = 200$  ns,  $p_f = 8$  ms, for a range of fault rates

| Page fault rate $p$  | EAT                | Slowdown vs. fault-free (200 ns) |
|----------------------|--------------------|----------------------------------|
| 0 (no faults)        | 200 ns             | 1×                               |
| 0.0001 (1 in 10,000) | $\approx 1,000$ ns | $\approx 5\times$                |
| 0.001 (1 in 1,000)   | $\approx 8,200$ ns | $\approx 41\times$               |

**This is why keeping the fault rate near zero is non-negotiable.** If demand paging must cost close to nothing in practice, the fault rate has to stay a tiny fraction of a percent — exactly the pressure that motivates Lecture 26's page-replacement algorithms.

# Copy-on-Write (COW)

**Copy-on-write** delays the copy `fork()` would otherwise do until it's actually needed. Parent and child initially share the exact same physical pages, every shared page marked read-only.

Share first, copy only on a write

## **`fork()` creates the child**

Parent and child share every physical page, all marked read-only

## **Both processes run and read freely**

No copying has happened yet — reads need no privacy

## **One process writes to a shared page**

Read-only protection triggers a fault — the OS copies just that one page

# Why Copy-on-Write Is a Huge Win

- **fork()** is extremely often followed almost immediately by **exec()**. The child forks only to then replace its entire address space with a different program.
- Under copy-on-write, that sequence never copies anything at all: the child shares the parent's pages for the brief interval before **exec()** discards that address space wholesale.
- The “full copy” that naive **fork()** would have performed was pure wasted work from the very start.

**Which pages actually get copied?** Only the handful of pages either process writes to before sharing ends (by exiting, or by **exec()**) are ever duplicated. COW doesn't eliminate copying in general — it stops paying for copies that would never have been needed.

# Key Takeaways

- **Virtual memory** separates logical address space from physical memory, letting a process run with only part of itself resident.
- **Pure demand paging** starts a process with zero resident pages and loads every page exactly once, on first reference.
- **Page-fault handling** is a precise sequence: trap, distinguish illegal from valid-but-unloaded, find a frame, read the page in, update the page table, restart the instruction.
- **EAT** shows a tiny fault rate devastates performance:  $p = 0.001$  alone produced a 41× slowdown.
- **Copy-on-write** shares pages read-only and copies only on first write — `fork()` immediately followed by `exec()` costs no copying at all.